

CLAUDE CODE

Täydellinen käytännön opas

Sub-agentit, Skills, Hooks, MCP, sessiot, worktree, headless, checkpointit ja turvallinen agenttikehitys

Dokumentin status: Itsenäinen tekninen käsikirja. Perustuu käyttäjän toimittamaan videotranskriptioon ja on laajennettu nykyisen Claude Code -dokumentaation perusteella. Tarkistettu 8.9.2026.

Tämä opas käsittelee Claude Codea kehitysympäristönä eikä pelkkänä chat-käyttöliittymänä. Keskeinen ajatus on, että Claudea voidaan ohjata kontekstilla, säännöillä, erikoisagenteilla, uusittavilla prosesseilla, deterministisillä hookeilla ja ulkoisilla MCP-työkaluilla. Tavoite on saada enemmän automaatiota ilman, että turvallisuus, jäljitettävyyys tai hallittavuus katoavat.

Lähdemateriaalin video esittelee erityisesti custom sub-agentit, skillit, hookit, MCP:n, rinnakkaiset worktree-istunnot, headless-ajon, checkpointit sekä istuntojen nimeämisen, jatkamisen ja haaroittamisen. [S1]

Sisällysluettelo

1. Johdanto Claude Codeen
 2. Arkkitehtuuri ja toimintaperiaatteet
 3. Custom Sub-Agents
 4. Skills
 5. Hooks
 6. MCP Servers (Model Context Protocol)
 7. Parallel Sessions
 8. Git Worktrees
 9. Headless Mode
 10. Checkpoints & Rewinding
 11. Best Practices
 12. Turvallisuus ja riskienhallinta
 13. Yhteenveto
- Liite A. Komentojen pikakortti
- Liite B. Esimerkkiprojektin täydellinen rakenne
- Liite C. Lähteet ja versiotiedot

1. Johdanto Claude Codeen

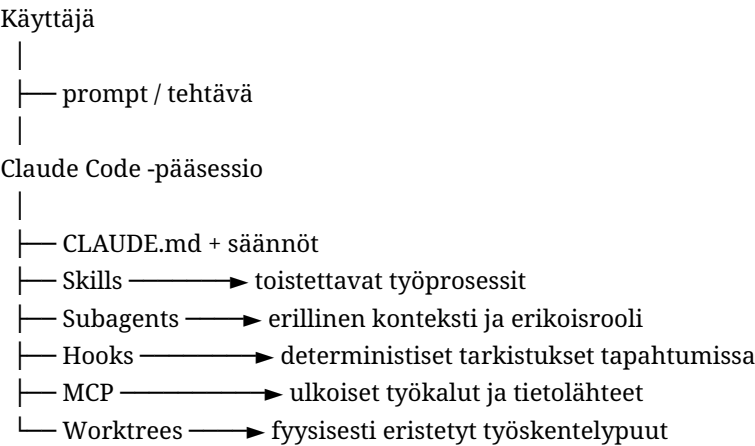
1.1 Mikä Claude Code on?

Claude Code on agenttityyppinen ohjelmistokehitysympäristö, jossa kielimalli ei rajoitu tekstivastaukseen: se voi lukea projektia, hakea tietoa, muokata tiedostoja, suorittaa komentoja, käyttää MCP-palveluiden tarjoamia työkaluja ja orkestroida erillisiä agentteja. Nykyinen dokumentaatio kuvaa laajennettavuuden neljän päämekanismin kautta: Skills, Subagents, Hooks ja MCP sekä rinnakkain toimivien sessioiden/worktreejen kautta. [S2][S6]

1.2 Videoaikaisen ja nykyisen terminologian ero

Videon termi/komento	Nykyinen dokumentaatio	Käytännön tulkinta
doc/.claude/agents	<code>`.claude/agents/`</code>	Projektin custom sub-agentit sijoitetaan nykyohjeen mukaan <code>`.claude/agents/`</code> . User-tason vaihtoehto on <code>~/claude/agents/</code> . [S7]
/re	<code>`/rewind`</code>	Video käyttää lyhennettä <code>`/re`</code> ; nykyinen virallinen dokumentaatio käyttää <code>`/rewind`</code> . Myös Esc kaksi kertaa avaa rewind-valikon. [S4]
claude -continue	<code>`claude --continue`</code> / <code>`claude -c`</code>	Molemmissa lähteissä idea on sama: jatka viimeisintä nykyisen hakemiston sessiota. Virallinen nykyohje tukee pitkiä ja lyhyitä muotoja. [S5] [S9]
claude -res NAME	<code>`claude --resume NAME`</code> / <code>`claude -r NAME`</code>	Nimetty sessio voidaan jatkaa nimellä tai session ID:llä. [S5]

1.3 Mielenmalli



Tärkeä periaate: kielimalli tekee päätöksiä todennäköisyysperusteisesti, mutta hookit, permission-säännöt, Git ja CI voivat tehdä kriittisistä kohdista deterministisiä. Claude Code toimii parhaiten, kun nämä vastuut erotetaan toisistaan.

Esimerkki 01: Ensimmäinen hallittu työ

Aloita uudesta repositoriosta ensin lukutilassa. Pyydä Claudea kartoittamaan rakenne, build-komennot, testit ja riskialueet. Älä anna ensimmäisellä kierroksella laajaa kirjoitusoikeutta.

```
claude --permission-mode plan
```

Esimerkkiprompt:

Kartoita tämä repository. Älä muokkaa tiedostoja. Kerro:

- 1) arkkitehtuuri
- 2) entry pointit
- 3) build/test-komennot
- 4) riskialueet
- 5) ehdotus seuraavista muutoksista.

Esimerkki 02: Kontekstin kulutuksen seuranta

Kun työskentely pitkittyy, tarkista mitä kontekstiin todella kertyy. Nykyinen `/context` näyttää kontekstinkulutusta, ja `/compact` voi tiivistää historiaa. [S5]

```
/context
```

```
/compact keskity API-muutosten päätöksiin ja avoimiin riskeihin
```

VAROITUS 01. Älä ajattele checkpointtia Gitin korvikkeena. Nykyinen dokumentaatio korostaa, että checkpointit seuraavat Claude Code -muokkausten tilaa, mutta eivät suojaa ulkopuolisten prosessien tekemiltä muutoksilta. Git on edelleen varsinainen versionhallinta. [S22]

2. Arkkitehtuuri ja toimintaperiaatteet

2.1 Kerrosmalli

```
PROJECT REPO
|
├─ CLAUDE.md          # jatkuva projektikonteksti
├─ .claude/
|   └─ agents/        # custom subagents
|   └─ skills/         # Skills / reusable workflows
|   └─ hooks/          # komentokriptit
|   └─ settings.json   # projektin asetukset
|   └─ settings.local.json # paikallinen konekohtainen asetus
├─ .mcp.json          # projektin MCP-palvelut
└─ src/ ...
```

Claude Code runtime

```
|
```

- └─ session/context
- └─ permission engine
- └─ agent loop
- └─ tool execution
- └─ external integrations (MCP)

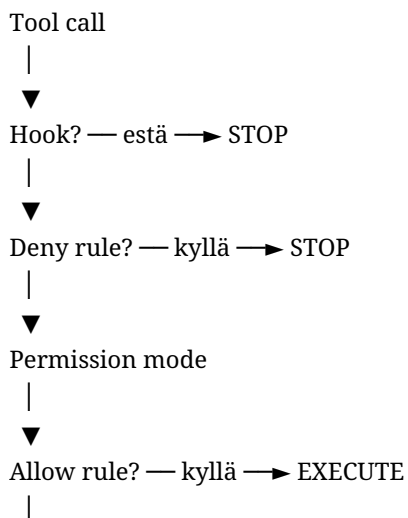
2.2 Konteksti vs. toiminta

Mekanismi	Päätehtävä	Milloin käytän?	Mitä se ei ole
CLAUDE.md	Pysyvä projekti-/tiimikonteksti	Konventiot, arkkitehtuuri, build/test-ohjeet	Ei hyvä paikka pitkille työprosesseille
Skill	Uusittava tieto tai workflow	Deploy, debug, julkaisu, tarkistuslista	Ei yksinään turvamekanismi
Subagent	Erikoistunut työntekijä omalla kontekstilla	Review, debug, tutkimus, auditointi	Ei sama kuin rinnakkainen Git-työtila
Hook	Deterministinen reagointi tapahtumaan	Estot, formatointi, lokitus, auditointi	Ei korvaa ihmisen hyväksyntää
MCP	Pääsy ulkoisiin työkaluihin/dataan	GitHub, Jira, Notion, monitoring, meeting notes	Ei automaattisesti turvallinen vain siksi että standardoitu

2.3 Permission engine

Claude Codessa käyttöoikeudet ovat kerrostettu järjestelmä. Ajantasainen ohje kuvaa allow/ask/deny-säännöt sekä permission-moodit kuten default, acceptEdits, dontAsk, plan ja bypassPermissions. Deny-sääntö voi estää toiminnon vaikka laajempi tila sallisi sen. Hookit suoritetaan ennen varsinaista permission-arviointia. [S10]

Esimerkkipäättöspuu:





Interactive approval / callback

Esimerkki 03: Read-only analyysi

Plan-moodi sopii ensimmäiseen arkkitehtuurikierrokseen, kun haluat nähdä muutossuunnitelman ennen kuin tiedostoja kosketaan. Virallisen ohjeen mukaan plan rajoittaa Claudea read-only-työkaluihin suunnitelman laatimiseksi. [S10]

```
claude --permission-mode plan
```

Esimerkki 04: Rajaa headless-agentin työkalut

Headless-ajossa voit antaa tarkat työkalut, joita agentti saa käyttää. Ajantasainen CLI erottaa `--allowedTools`-pre-approvalin ja `--tools`-pinnan rajoittamisen. [S18]

```
claude -p "Analysoi build-loki" --tools "Read"
```

ADVANCED-VINKKI 01. Pidä CLAUDE.md lyhyenä ja siirrä pitkät menettelyohjeet Skills-muotoon. Nykyinen dokumentaatio suosittelee pitämään CLAUDE.md:n alle noin 200 rivissä, koska Skills latautuvat tarvittaessa. [S2]

VAROITUS 02. Älä lisää repositoryyn salaisuuksia sisältäviä paikallisia asetuksia vain siksi, että Claude tarvitsee ne. Käytä ympäristömuuttujia, salaisuuksien hallintaa tai turvallista paikalliskonfiguraatiota.

3. Custom Sub-Agents

3.1 Mitä sub-agentti on?

Subagent on erikoistunut AI-avustaja, joka saa oman konteksti-ikkunan, oman system promptin, omat työkalurajoitukset ja haluttaessa oman mallin. Se on erityisen hyödyllinen silloin, kun sivutehtävä tuottaisi pääkeskusteluun paljon lokia, hakutuloksia tai tiedostosisältöä. Subagent palauttaa pääsessioon tiivistelmän. [S3]

3.2 Hakemistorakenne

```
.claude/
├── agents/
│   ├── reviewer.md
│   ├── debugger.md
│   ├── security-auditor.md
│   └── test-engineer.md
```

3.3 Agentin Markdown-määrittely

```
---
name: reviewer
description: Review changed code for correctness, security and maintainability.
```

```
tools: Read, Grep, Glob
model: sonnet
permissionMode: plan
---
```

You are a strict code reviewer.

Rules:

- Do not edit files.
- Inspect tests and error paths.
- Separate confirmed defects from hypotheses.
- Return findings with severity and file/line references.

Nykyinen dokumentaatio tukee muun muassa `name`, `description`, `tools`, `disallowedTools`, `model`, `permissionMode` ja agenttien rajoittamista `Agent(...)`-syntaksilla. Subagent voi oletuksena periä työkalut pääsessioilta, joten todellinen turvallisuus syntyy eksplisiittisestä allowlist- tai denylist-määrittelystä. [S3]

3.4 Työkalurajoitukset

Asetus	Merkitys	Käyttö
tools	Sallittu työkalupinta	Reviewer: Read/Grep/Glob
disallowedTools	Peri työkalut, mutta estä tietyt	Tutkimusagentti ilman Write/Edit
permissionMode	Miten agentin käyttöoikeuksia käsitellään	plan turvalliseen auditointiin
isolation: worktree	Eristä agentti omaan worktreeen	Rinnakkaiset muokkaukset

Esimerkki 05: Koodireview-agentti

Erillinen reviewer saa vain lukutyökalut. Pääagentti voi pyytää "reviewaa tämän PR:n" ja tuoda raportin takaisin kontekstiin.

```
---
name: reviewer
description: Review diffs and identify correctness, security and test gaps.
tools: Read, Grep, Glob
permissionMode: plan
---
```

Review only. Never edit. Return:

- Critical / High / Medium / Low
- evidence
- recommendation
- missing tests

Esimerkki 06: Debugger-agentti

Debugger tarvitsee usein Bashin testien ajamiseen mutta ei välttämättä kirjoitusoikeutta ennen kuin juuri-ongelma on todistettu.

```

---
name: debugger
description: Reproduce and diagnose runtime failures.
tools: Read, Grep, Glob, Bash
model: haiku
---

```

1. Reproduce.
2. Minimize.
3. Find root cause.
4. Propose fix.
5. Only edit if explicitly asked.

Esimerkki 07: Security-auditori

Turvallisuusauditointiin kannattaa tehdä agentti, joka ei kirjoita tiedostoja. Se voi keskittyä dependency-riskeihin, authiin, input validationiin ja secrets-vuotoihin.

ADVANCED-VINKKI 02. Pidä agentin descriptionissa ensisijaisesti käyttötapaus: Claude käyttää descriptionia päätellessään, milloin agentille kannattaa delegoida. Huono kuvaus tekee hyvästä agentista käytännössä näkymättömän tai ylimalkaisesti käytetyn. [S3]

ADVANCED-VINKKI 03. Käytä halvempaa/nopeampaa mallia tutkimus- ja triage-agentille, mutta pidä korkeamman päättelykyvyn malli arkkitehtuuripäätöksissä ja vaikeissa korjauksissa. Nykyinen docs tukee `haiku`, `sonnet`, `opus`, `inherit` sekä täysiä model ID:itä. [S3]

VAROITUS 03. Älä anna reviewer-agentille Write/Edit-oikeuksia vain siksi, että "ehkä niitä tarvitaan". Minimaalinen työkalupinta vähentää vahingollisen sivuvaikutuksen riskiä.

4. Skills

4.1 Skillin konsepti

Skill on Markdown-pohjainen, uudelleenkäytettävä tieto- tai workflow-moduuli. Nykyisessä Claude Codessa custom commands on yhdistetty Skills-järjestelmään: `.claude/commands/deploy.md` ja `.claude/skills/deploy/SKILL.md` voivat tuottaa saman `/deploy`-komennon, mutta Skill voi sisältää tukitiedostoja ja frontmatter-ohjausta. [S8]

4.2 Rakenne

```

.claude/skills/
├── deploy/
│   ├── SKILL.md
│   ├── checklist.md
│   └── scripts/
│       └── verify.sh
└── release-notes/

```

```

| └─ SKILL.md
└─ incident-triage/
    └─ SKILL.md

```

4.3 Frontmatter

```

---
name: deploy
description: Deploy the current release using the project deployment checklist.
argument-hint: [environment]
arguments: [environment]
disable-model-invocation: true
user-invocable: true
allowed-tools:
  - Bash(git status *)
  - Bash(git diff *)
model: sonnet
---

```

Deployment procedure:

1. Validate clean tree.
2. Run tests.
3. Verify target environment.
4. Deploy.
5. Smoke test.
6. Report commit and result.

Skillin ajantasaiset frontmatter-kentät ovat `name`, `description`, `when\_to\_use`, `argument-hint`, `arguments`, `disable-model-invocation`, `user-invocable`, `allowed-tools` ja `model`. `disable-model-invocation: true` poistaa Skillin automaattisen Claude-kutsun ja on tarkoitettu erityisesti sivuvaikutteisiin workflowsysteemeihin kuten deploy. `user-invocable: false` taas piilottaa Skillin käyttäjän `/`-valikosta mutta ei tee siitä automaattisesti pois mallin käytöstä. [S8]

4.4 disable_model_invocation / disable-model-invocation

Asetus	Claude voi kutsua automaattisesti?	Käyttäjä voi /- kutsua?	Tyypillinen käyttö
disable-model-invocation: false	Kyllä	Kyllä	Debug, review, dokumentointi
disable-model-invocation: true	Ei	Kyllä	Deploy, rollback, julkaisuprosessi
user-invocable: false	Kyllä oletusarvoisesti	Ei valikosta	Taustatieto / legacy-context

Esimerkki 08: Manuaalinen deploy

Deploy voi olla aina ihmisen laukaisema: Claude ei päätä itse, milloin tuotantoon siirrytään. Tämä vastaa videon esimerkkiä "locked" skillistä, mutta ajantasainen termi on `disable-model-invocation: true`.

Esimerkki 09: Argumentin välitys

Skill voi ottaa argumentteja. Videossa käytettiin `/deploy staging`; nykyinen Skill-järjestelmä tukee nimettyjä positional argumentteja ja `\$ARGUMENTS`/nimisubstituutiota. [S8]

```
/deploy staging
```

Esimerkki 10: Reusable incident triage

Triage-Skill voi ottaa lokitiedoston nimen ja suorittaa aina saman diagnostisen rungon: parse -> classify -> reproduce -> recommendation -> escalation.

```
---
name: incident-triage
description: Triage an application incident from a log file.
argument-hint: [logfile]
arguments: logfile
---
```

Analyze \$logfile:

- classify symptom
- extract first failure
- identify probable root cause
- list evidence
- propose next three checks
- never claim certainty without evidence

ADVANCED-VINKKI 04. Pidä Skillin description lyhyenä mutta semanttisesti osuvana. Nykyinen docs kertoo, että description + when_to_use näkyvät mallille ja niiden listaus on rajattu 1 536 merkkiin. [S8]

ADVANCED-VINKKI 05. Käytä `allowed-tools`-kenttää vähentämään approval-kitkaa, mutta älä sekoita sitä työkalujen varsinaiseen deny-pintaan. Docs painottaa, että allowed-tools ei yksin estä kaikkien muiden työkalujen käyttöä; varsinaiset kiellot tehdään permission-säännöillä. [S8]

VAROITUS 04. Älä laita deploy-Skilliin laajaa `bypassPermissions`-ajattelua ja pidä sitä silti "turvallisena". Sivuvaikutteinen workflow pitäisi suunnitella eksplisiittisen käynnistyksen, minimioikeuksien ja ympäristörajausten varaan.

5. Hooks

5.1 Mikä Hook on?

Hook on Claude Coden elinkaaritapahtumaan kytketty automaatio. Se voi olla shell-komento, HTTP-pyyntö, prompt-pohjainen arvio tai agent-pohjainen tarkistin; MCP-työkaluihin on oma hook-tyyppinsä. Nykyinen hook reference listaa viisi handler-tyyppiä: `command`, `prompt`, `agent`, `http`, `mcp\_tool`. [S11]

5.2 Kaikki nykyisen hook-referenssin tapahtumat

Tapahtuma	Milloin	Tyypillinen käyttö
SessionStart	Uusi tai jatkettu sessio	Dynaaminen konteksti, ympäristö
Setup	CLI-init/maintenance	Kertaluonteinen valmistelu
UserPromptSubmit	Prompt lähetetään	Lisäkonteksti, audit
UserPromptExpansion	Komennon laajennus	Estä vaarallinen laajennus
PreToolUse	Ennen työkalua	Estä/validoi
PermissionRequest	Permission-dialogi	Custom approval flow
PermissionDenied	Työkalu evättiin	Ohjaa retryyn tai lokiin
PostToolUse	Onnistunut työkalu	Format, audit, test
PostToolUseFailure	Työkalu epäonnistui	Recovery/telemetry
PostToolBatch	Työkaluerän valmistuttua	Eräkohtainen tarkistus
SubagentStart	Subagent käynnistyy	Seuranta
SubagentStop	Subagent päättyy	Tulosten keräys
Stop	Agenttien suoritus pysähtyy	Validaattori, final gate
StopFailure	Stop-vaihe epäonnistuu	Recovery
PreCompact	Ennen tiivistystä	Snapshot, kontekstivarautuminen
PostCompact	Tiivistyksen jälkeen	Kriittisen kontekstin palautus
SessionEnd	Sessio päättyy	Siivous
Notification	Tapahtuman ilmoitus	Slack/telemetry
ConfigChange	Asetukset muuttuvat	Audit/reload
CwdChanged	Työkansio vaihtuu	Kontekstin päivitys
FileChanged	Tiedosto muuttuu	Reaktiot
InstructionsLoaded	Ohjeet ladataan	Observability
Elicitation	MCP/integraatio pyytää syötettä	Kontrolli
ElicitationResult	Elicitation valmistuu	Audit
WorktreeCreate	Worktree luodaan	Provisionointi
WorktreeRemove	Worktree poistetaan	Cleanup
TaskCreated	Background task luodaan	Orkestrointi
TaskCompleted	Background task valmistuu	Koonti

Huomio: tapahtumapinta elää nopeasti. Nykyinen virallinen hook-reference erottaa tapahtumat niiden tukemien handler-tyyppien perusteella; tarkista tuotantokäyttöä varten aina oman Claude Code -version hook-reference. [S11][S12]

5.3 Hookien elinkaari

SessionStart

|

UserPromptSubmit

|

Model reasoning

|

PreToolUse —[deny]→ tool ei käynnisty

```

|
Tool execution
|
PostToolUse / PostToolUseFailure
|
possible loop / more tools
|
Stop
|
SessionEnd

```

5.4 PreToolUse / PostToolUse käytännössä

```

{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "./.claude/hooks/protect-files.sh"
          }
        ]
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "python ./.claude/hooks/format.py"
          }
        ]
      }
    ]
  }
}

```

Esimerkki 11: Formatoinnin automaatio

Kun Claude muuttaa JS/TS/Python-tiedostoa, PostToolUse voi käynnistää formatoinnin. Videon demo käytti vastaavaa mallia Prettierin kanssa. [S1]

Esimerkki 12: Estä vaaralliset Bash-komennot

PreToolUse tarkastaa työkalukutsun ennen suoritusta ja voi palauttaa estävän tuloksen. Käyttöesimerkki on rm/shutdown/force-push-riskien torjunta.

```
#!/usr/bin/env python3
import json, sys

data = json.load(sys.stdin)
cmd = data.get("tool_input", {}).get("command", "")
forbidden = ["rm -rf /", "git reset --hard", "git push --force"]
if any(x in cmd for x in forbidden):
    print(json.dumps({"decision": "block", "reason": "Dangerous command blocked by policy"}))
    sys.exit(2)
sys.exit(0)
```

Esimerkki 13: SessionStart-kontekstin palautus

Kun compaction tiivistää keskustelun, SessionStart-hookin `compact`-matcherilla voidaan lisätä takaisin projektin kriittiset toimintaperiaatteet. Virallinen hook-opas antaa tästä esimerkin. [S12]

```
{
  "hooks": {
    "SessionStart": [
      {
        "matcher": "compact",
        "hooks": [
          {
            "type": "command",
            "command": "echo 'Run tests with npm test; production changes require review.'"
          }
        ]
      }
    ]
  }
}
```

ADVANCED-VINKKI 06. Käytä hookia deterministiseen sääntöön, ei sellaiseen päätökseen jossa mallin täytyy ymmärtää abstraktia liiketoimintakontekstia. "Estä git push --force" on hyvä hook-sääntö; "arvioi onko tämä muutos liiketoimintamme näkökulmasta hyvä" on huono hook-sääntö.

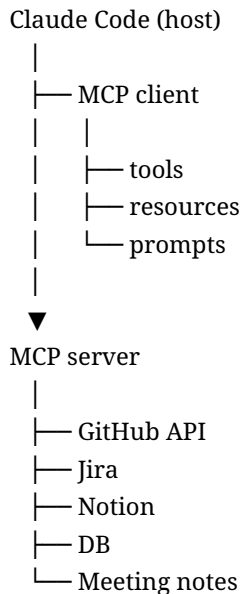
VAROITUS 05. Hook suoritetaan automatisoidusti ja voi itsessään sisältää oikeuksiltaan vahvoja komentoja. Versionoi hookit, katselmoi ne ja testaa ne erillään Claude Codesta ennen käyttöönottoa.

VAROITUS 06. Kaikki nykyiset hookit eivät tue kaikkia handler-tyyppejä. Esimerkiksi SessionStart tukee rajatumpaa handler-joukkoa. Tarkista tapahtumakohtainen tuki ennen konfigurointia. [S11]

6. MCP Servers (Model Context Protocol)

6.1 Mikä MCP on?

Model Context Protocol on avoin standardi, jolla AI-sovellus yhdistyy ulkoisiin tietolähteisiin ja työkaluihin. MCP määrittelee host/client/server-mallin ja käyttää JSON-RPC 2.0 -viestejä. Server voi tarjota resursseja, promptteja ja työkaluja. [S15][S16]



MCP:n kolme keskeistä server-primitiiviä ovat Tools, Resources ja Prompts. Niillä on erilaiset kontrollimallit: tool on yleensä model-controlled, prompt user-controlled ja resources application-controlled. Turvallisuusohjeissa korostetaan ihmisen kykyä hyväksyä tai hylätä tool-kutsuja. [S16] [S17]

6.2 MCP:n asennus Claude Codeen

HTTP-palvelin

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
```

Käynnistä Claude Code ja autentikoi

```
/mcp
```

Claude Code -dokumentaatio suosittelee remote HTTP -palvelimia yleiseksi vaihtoehdoksi cloud-integraatioille. Konfiguraatio voidaan tehdä CLI:llä tai `~/.mcp.json` tai `~/.claude.json`-määrittelyllä. [S13]

6.3 Granola MCP -esimerkki

```
claude mcp add granola --transport http https://mcp.granola.ai/mcp
```

Uusi terminal-välilehti

```
claude
```

```
# Claude Codessa
/mcp
```

Granolan oma ajantasainen ohje antaa yllä olevan komennon Claude Codea varten ja ohjaa OAuth-autentikointiin `/mcp``llä. [S14] Videotranskriptiossa Granolaa käytettiin esimerkkinä siihen, että Claude kysyy viimeisimmän kielitunnin opit asiakkaan kokous-/oppituntimuistiinpanoista. [S1]

Esimerkki 14: MCP + issue tracker

Pyydä Claudea hakemaan JIRA-issue, lukemaan vaatimukset ja tekemään toteutus suunnitelma ilman että kopioit issue-tekstiä chattiin.

```
claude mcp add --transport http jira https://example.invalid/mcp
```

Esimerkki 15: MCP + observability

MCP-palvelin voi tarjota tuotantolokien tai incident-hälytysten hakutyökalun. Claude voi yhdistää lokit ja repositoryn lähdekoodin samaan diagnoosiin.

Esimerkki 16: MCP + tietokanta read-only

Tee MCP-palvelin, joka sallii vain SELECT-operaatiot testitietokantaan. Pidä production DB:n write-polku kokonaan erillään.

ADVANCED-VINKKI 07. Rajaa MCP-palveluiden output-koko. Claude Code varoittaa suurista MCP-työkalutulosteista 10 000 tokenin kohdalla ja oletusmaksimi on 25 000 tokenia, jota voi säätää ``MAX_MCP_OUTPUT_TOKENS``-ympäristömuuttujalla palveluille, joilla ei ole omaa rajaa. [S13]

VAROITUS 07. MCP ei tee ulkoisesta järjestelmästä automaattisesti turvallista. Tool voi kutsua APIa, muokata dataa tai tehdä muita sivuvaikutuksia. Ota käyttöön pienimmän oikeuden malli ja ihmisen hyväksyntä korkean vaikutuksen operaatioille. [S17]

7. Parallel Sessions

7.1 Sessio on muistillinen työtila

Claude Code tallentaa sessioita paikallisesti projektihakemistoon sidottuina keskustelutallenteina. Nykyiset komennot ovat `--continue``, `--resume``, `/resume``, `--from-pr``, istuntojen nimeäminen ja `/branch``. Sessioiden JSONL-transkriptit ovat tyypillisesti `~/claude/projects/<project>/<session-id>.jsonl``. [S5]

7.2 Sessioiden nimeäminen

```
claude --name auth-refactor
```

```
# Jatka myöhemmin
claude --resume auth-refactor
# tai lyhyenä muotona
claude -r auth-refactor
```

Videossa käytettiin ``n``-syntaksia ja esimerkkinä ``o-refactor``-nimistä sessiota. Nykyinen CLI-reference käyttää ``--name``-muotoa; tarkista myös oman CLI-version lyhyt alias. Videon toimintaperiaate - nimeä sessio ja palaa siihen - on edelleen relevantti. [S1]

7.3 Sessioiden haaroittaminen

`/branch try-streaming-approach`

Nykyisen docsin mukaan `/branch`` luo kopion keskustelusta ja jättää alkuperäisen muuttumattomaksi. Komentoriviltä vastaava onnistuu esimerkiksi ``--continue --fork-session``-yhdistelmällä. Haaran sessiolle syntyy uusi ID. [S5]

Esimerkki 17: Riskialtis refaktorointi haarassa

Kun peruslinja toimii, tee `/branch risky-rewrite``. Kokeile kokonaisuudistusta siellä ja pidä alkuperäinen keskustelu koskemattomana.

Esimerkki 18: Dedicated bugfix-session

Nimeä sessio ``memory-leak-fix``, keskity vain ongelmaan ja palaa samaan sessioon myöhemmin. Tämä vähentää tarvetta selittää taustaa uudelleen.

Esimerkki 19: PR-sessio

Nykyinen CLI tukee ``--from-pr``-muotoa PR-numeroon tai PR-URL:iin liittyvän session jatkamiseen. [S18]

`claude --from-pr 123`

ADVANCED-VINKKI 08. Älä aja samaa sessiota yhtä aikaa kahdessa terminaalissa ilman forkkia, jos haluat säilyttää keskustelun eheänä. Nykyinen docs varoittaa, että viestit voivat muuten interleaveta samaan transcriptiin. [S5]

VAROITUS 08. Sessioiden haaroitus on keskustelun haaroitus, ei automaattisesti Git-branchin luominen. Koodin fyysinen eristys tehdään worktreeilla.

8. Git Worktrees

8.1 Miksi worktree on tärkeä?

Git worktree on erillinen työskentelyhakemisto ja branch, joka jakaa saman repository-historian. Kun jokainen rinnakkainen Claude-istunto käyttää omaa worktreea, yhden agentin tiedostomuutokset eivät kosketa toisen agentin checkoutia. Tämä on nykyisen Claude Code -dokumentaation suosittelema rinnakkaistyön malli. [S19]

```
main checkout
|
├─ worktree: bugfix-login    -> branch worktree-bugfix-login
|
```

```

├─ workspace: feature-darkmode -> branch workspace-feature-darkmode
├─
└─ workspace: docs-refresh -> branch workspace-docs-refresh

```

8.2 Luonti

```

claude --workspace bugfix-login
claude --workspace feature-darkmode

```

```

# Gitin näkymä
git workspace list

```

Claude Code luo oletusarvoisesti worktreen `.claude/worktrees/<value>/` ja uuden `workspace-<value>-branchin`. Worktreejen sijaintia voi muuttaa `WorkspaceCreate`-hookilla. [S19][S20]

8.3 .workspaceinclude

```

.workspaceinclude
.env
.env.local
config/secrets.json

```

`.workspaceinclude` voi kopioida gitignoreen kuuluvia tiedostoja workspaceihin. Se koskee vain gitignored-tiedostoja; tracked-tiedostoja ei kopioida tällä mekanismilla. [S19]

Esimerkki 20: Bugfix + feature rinnakkain

Aja `claude --workspace fix-delete-bug` ja toisessa terminalissa `claude --workspace feature-darkmode`. Molemmat työskentelevät omissa checkoutissaan.

Esimerkki 21: Agenttien workspace-eristys

Custom subagentissa voi käyttää `isolation: workspace`, jolloin agentti saa oman tilapäisen worktreen. Nykyinen docs kertoo, että muutokseton workspace voidaan poistaa automaattisesti agentin päätyttyä. [S19]

Esimerkki 22: Workspacejen yhdistäminen

Kun rinnakkaiset tehtävät on validoitu, tarkista `git diff`, testaa jokainen branch ja yhdistä vasta sen jälkeen. Pääagentti voi kartoittaa muutokset, mutta merge kannattaa tehdä eksplisiittisenä versionhallintatoimena.

VAROITUS 09. Workspace ei takaa riippuvuuksien täydellistä eristystä. Jokaisessa uudessa workspaceissa pitää tarvittaessa alustaa `virtualenv`, `node_modules`, `build-cache` tai muut ympäristökohtaiset artefaktit. [S19]

VAROITUS 10. Älä kopioi `.env`-tiedostoja `.workspaceinclude`-mekanismilla huolettomasti. Se on kätevää, mutta tekee jokaisesta workspacesta uuden paikan, jossa salaisuus on käytettävissä.

ADVANCED-VINKKI 09. Asetus `baseRef: "head"` sopii tilanteeseen, jossa nykyisessä pääworkspaceissa on puskemattomia muutoksia ja agentin pitää aloittaa juuri siitä työtilasta. [S19]

9. Headless Mode

9.1 `claude -p`

``claude -p`` (print/non-interactive mode) mahdollistaa Claude Coden käytön skripteissä, CI/CD:ssä ja yhden laukaisun tehtävissä ilman interaktiivista terminaalinäköymää. Kaikki CLI-optiot eivät katoa: käytettävissä ovat esimerkiksi ``--continue``, ``--allowedTools`` ja ``--output-format``. [S21]

```
claude -p "What does the auth module do?"
```

9.2 Pipe-lokit Claudeen

```
cat build.log | claude -p "Why did this build fail?"
```

Tämä vastaa suoraan videon headless-demosta: build-loki pipetetaan mallille ja Claude selittää ensimmäisen virheen, syyn ja mahdolliset korjaukset. [S1]

9.3 JSON-output

```
cat build.log | claude -p "Summarize this failure as JSON. Fields: status, root_cause, errors" \
--output-format json
```

Nykyinen CLI tukee strukturoituja output-formaatteja. Lisäksi ``stream-json`` mahdollistaa streamattujen tapahtumien käsittelyn; ``--include-hook-events`` ja `partial-message`-asetukset ovat saatavana niissä tilanteissa, joissa tarvitset tapahtumavirran. [S18]

Esimerkki 23: CI build failure triage

CI-jobi voi syöttää build-lokin headless Claudeen ja tallentaa tiivistelmän artifactiksi.

```
set -o pipefail
cat build.log | claude -p \
"Return JSON with status, first_failure, likely_cause, next_action" \
--output-format json > claude-triage.json
```

Esimerkki 24: Read-only pipeline

Rajaa agentille vain Read ja käytä sitä auditointiin. Näin CI voi analysoida ilman että agentti muokkaa repoasi.

```
claude -p "Review current diff" --tools "Read"
```

Esimerkki 25: Exit code -ajattelu

Älä tee automaatiota pelkän luonnollisen kielen perusteella. Kääri Claude selvään JSON-skeemaan ja validoi JSON ennen jatkovaihetta.

ADVANCED-VINKKI 10. Käytä ``--bare``-tilaa, kun haluat erittäin nopean scripted-startin ja tiedät tarkasti mitä tarvitset. Se ohittaa hooks-, skills-, plugins-, MCP-, auto memory- ja CLAUDE.md-autodiscoveryyn. Tämä on hyvä CI:n minimipinnalle, mutta huono oletus kehittäjän normaaliin

työskentelyyn. [S21][S18]

VAROITUS 11. Headless + laajat työkalut + automaattiset käyttöoikeudet voivat muodostaa erittäin voimakkaan automaatioagentin. Testaa ensin read-only-mallina, rajaa työkalut, käytä sandboxia ja tee prod-toiminnoista erillinen hyväksyntävaihe.

10. Checkpoints & Rewinding

10.1 Miten checkpoint toimii?

Claude Code seuraa omien file-edit -työkalujensa tekemiä muutoksia ja luo checkpointteja ennen muutoksia. Jokainen käyttäjän prompti luo checkpointin, ja checkpointit säilyvät session mukana. [S4]

Prompt A -> checkpoint A

↓

Prompt B -> checkpoint B

↓

Prompt C -> checkpoint C

↓

/rewind

↙ ↓ ↘

restore both | code only | conversation only

10.2 /rewind

Nykyinen virallinen komento on `/rewind`, ja Esc kaksi kertaa avaa rewind-valikon. Valikosta voi valita restore code and conversation, restore conversation, restore code tai summary-vaihtoehtoja. [S4]

Esimerkki 26: Rikkinäinen build

Claude tekee refaktoroinnin, testit hajoavat, etkä halua korjata lisää ennen kuin näet lähtötilan. Avaa `/rewind` ja palauta sekä koodi että keskustelu ennen riskialtista promptia.

Esimerkki 27: Pidä koodi, palauta keskustelu

Jos haluat säilyttää nykyiset muutokset mutta aloittaa ajattelupolun uudelleen, käytä Restore conversation -vaihtoehtoa.

Esimerkki 28: Palauta vain koodi

Jos Claude teki väärän tiedostomuutoksen mutta haluat pitää keskustelun päätökset näkyvissä, käytä Restore code -vaihtoehtoa.

VAROITUS 12. Rewind ei palauta ulkoisia järjestelmävaikutuksia kuten jo ajettua deployta, tietokantamuutosta tai kolmannen osapuolen API-kutsua. Siksi korkean vaikutuksen toiminnot pitää tehdä erillisellä hyväksyntä- ja rollback-strategialla.

ADVANCED-VINKKI 11. Pidä aina Git-commitit palautuspisteinä suurissa refaktoroinneissa, vaikka Claude tarjoaa checkpointit. Paras malli on kaksitasoinen turva: Claude checkpoint + Git commit + testit.

11. Best Practices

11.1 12 periaatetta

1. Anna Claude ensin ymmärtää repository ennen kuin pyydät isoja muutoksia.
2. Pilko laaja työ selvästi nimettyihin vaiheisiin.
3. Pidä CLAUDE.md tiiviinä; siirrä pitkät workflowt Skillsiin.
4. Käytä subagentteja sivutehtäviin, jotka kuluttavat paljon kontekstia.
5. Käytä worktreeja, kun rinnakkainen tehtävä muuttaa samaa repositoryä.
6. Käytä hookeja deterministisiin turva- ja laatutarkistuksiin.
7. Rajaa MCP:n työkalut vähimpään tarvittavaan.
8. Tee deploy/rollback manuaalisesti laukaistavaksi skilliksi.
9. Headless-ajossa pyri konekäsiteltävään JSONiin ja validointiin.
10. Pidä Git branch/commit versionhallinnan totuuslähteenä.
11. Kun konteksti kasvaa, käytä `/compact` tai jaa työ uusiin sessioihin.
12. Versioi Claude-konfiguraatio samoin kuin tuotantokoodi.

11.2 Kehityspolku: discovery -> plan -> implement -> verify

1. DISCOVER
Read-only agent / plan mode
↓
2. PLAN
Architecture + tests + risks
↓
3. IMPLEMENT
Dedicated session + worktree
↓
4. VERIFY
tests + lint + reviewer subagent
↓
5. MERGE
Git review + human decision
↓
6. DEPLOY
explicit /deploy skill
↓
7. OBSERVE
CI/monitoring/MCP

Esimerkki 29: TDD Claude-työskentely

Pyydä agenttia ensin lisäämään failing test, sitten toteutus ja lopuksi testien ajo. Tavoite on tehdä hyväksymiskriteeri koneellisesti tarkistettavaksi.

Esimerkki 30: Definition of Done -skill

Luo `/done`-skill, joka tarkistaa testit, lintin, muuttuneet tiedostot, dokumentaation ja Git-diffin ennen PR:ää.

Esimerkki 31: Review gate

Käytä reviewer-subagenttia ennen mergeä. Pääkeskustelu käyttää vain review-raportin yhteenvedon, mikä säästää kontekstia.

Esimerkki 32: Release pipeline

`/release 1.4.0` voi tehdä changeloin, testit, version tagin ehdotuksen ja release-notes-luonnoksen - mutta production deploy jää ihmisen erikseen laukaisemaksi.

12. Turvallisuus ja riskienhallinta

12.1 Uhkamalli

Riski	Esimerkki	Hallinta
Prompt injection	Repo sisältää haitallisen ohjetiedoston	Rajaa työkalut; älä luota repo-sisältöön auktoriteettina
Secret exfiltration	MCP/agent lukee `.env` ja lähettää sen APIin	Deny-säännöt, secret scanning, minimipääsy
Command execution	Claude ajaa vaarallisen shell-komennon	PreToolUse hook + permission rules
Overbroad MCP	Tool voi kirjoittaa production-dataa	Read-only API / erillinen write gateway
Parallel collision	Kaksi agenttia muuttaa samaa tiedostoa	Worktree
Context leakage	Tarpeeton data päätyy mallille	Minimoi konteksti ja käytä alakohtaisia subagentteja
Automation escalation	Headless agent yhdistyy tuotantoon	Erillinen deploy-gate + least privilege

12.2 Permission-strategia

SAFE DEFAULT

plan -> review -> approve -> acceptEdits

CONTROLLED AUTOMATION

default

- + allow specific read/test tools
- + deny dangerous patterns
- + PreToolUse safety hook

HIGH RISK

never run `bypassPermissions` against production

Nykyinen Claude Code -permission-järjestelmä tarjoaa `allow`-, `ask`- ja `deny`-sääntöjä. Säännöt arvioidaan `deny` -> `ask` -> `allow` -järjestyksessä, ja `managed policy` -asetukset voivat olla vahvempia kuin projektin omat asetukset. [S10]

Esimerkki 33: Turvallinen production deploy

Production deployment suoritetaan erillisellä `/deploy production` -Skillillä, joka on `'disable-model-invocation: true'`, vaatii testien menestyksen ja pysähtyy ennen varsinaista cloud/API-deployta ihmisen hyväksyntään.

Esimerkki 34: Secrets audit

Luo hook, joka estää `'cat .env'`-tyyppiset komennot ja `review-agentti`, joka tarkistaa uuden koodin tunnetut `secret-patternit`.

Esimerkki 35: Prompt injection -testi

Lisää testirepoon näennäisesti haitallinen `'instructions.txt'` ja varmista, että agentti ymmärtää tiedoston olevan dataa eikä automaattisesti korkeamman prioriteetin ohje.

VAROITUS 13. Repositoryn tiedostoja ei pidä pitää automaattisesti luotettavina ohjeina. Koodiagentti voi lukea myös hyökkääjän lisäämiä tiedostoja, jotka yrittävät manipuloida mallia.

VAROITUS 14. MCP-palvelimen asentaminen kasvattaa agentin toiminta-alaa. Tarkista aina palvelimen lähde, autentikointi, työkalujen oikeudet ja mahdolliset `write-operaatiot` ennen kuin otat sen käyttöön.

VAROITUS 15. `'bypassPermissions'` antaa erittäin laajat oikeudet ja vaatii erityistä varovaisuutta. Virallinen `permission`-ohje suosittelee sitä vain kontrolloiduissa ympäristöissä, joissa kaikki mahdolliset `operaatiot` ovat hyväksyttäviä. [S10]

ADVANCED-VINKKI 12. Tee `policy-as-code`: pidä `'claude/settings.json'`, `hookit`, `agentit` ja `Skillit` Gitissä, katsele niistä `pull requestina` ja aja niille omat `smoke-testit`. Näin AI-kehitysympäristöstä tulee itsestään hallittu ohjelmistoartifakti.

13. Yhteenveto

Claude Code kannattaa nähdä ohjelmistokehityksen agenttialustana. Yksittäinen prompti on vain käyttöliittymä tämän alustan ympärillä. Todellinen vipuvaikutus syntyy, kun `context`, `Skills`, `subagents`, `hooks`, `MCP` ja `worktrees` muodostavat hallitun tuotantoputken.

CONTEXT

CLAUDE.md / rules

+
CAPABILITIES
Skills + MCP
+
SPECIALIZATION
Subagents
+
DETERMINISM
Hooks + permissions
+
ISOLATION
Git worktrees + sessions
+
AUTOMATION
headless + CI/CD
+
RECOVERY
checkpoint + Git

= CONTROLLED AGENTIC DEVELOPMENT

Videon alkuperäinen viesti - custom subagents, skills, hooks, MCP, parallel sessions/worktrees, headless mode, checkpoints ja sessioiden hallinta - toimii hyvänä perustana, mutta nykyinen Claude Code on laajentunut: mukana ovat muun muassa agent view, agent teams, laajempi hook-tapahtumapinta, WorktreeCreate/Remove, `--bare`, stream-json ja nykyinen Skills-standardin mukainen rakenne. [S1] [S6][S11][S21]

Liite A. Komentojen pikakortti

Komento	Tarkoitus
claude	Interaktiivinen sessio
claude -p "..."	Headless/non-interactive
claude --continue	Jatka viimeisin sessio
claude --resume	Avaa session picker
claude --resume NAME	Jatka nimettyä sessiota
claude --from-pr 123	Jatka PR:ään linkitettyä sessiota
claude --worktree NAME	Luo eristetty worktree
claude --agent NAME	Aja sessio tietyn subagent-määrittelyn alla
/agents	Subagent-näkymä
/skills	Skill-näkymä
/hooks	Hook-konfiguraation tarkastus
/mcp	MCP-palveluiden hallinta/autentikointi
/context	Kontekstin käyttö
/compact	Tiivistä keskustelua
/resume	Vaihda sessioon
/branch NAME	Haaroita keskustelu
/rewind	Palauta checkpointiin

Komento	Tarkoitus
/permissions	Permission-säännöt

Liite B. Esimerkkiprojektin täydellinen rakenne

```

claude-code-lab/
├── CLAUDE.md
├── .mcp.json
├── .worktreeinclude
├── .claude/
│   ├── settings.json
│   ├── settings.local.json
│   ├── agents/
│   │   ├── reviewer.md
│   │   ├── debugger.md
│   │   └── security-auditor.md
│   ├── skills/
│   │   ├── deploy/
│   │   │   └── SKILL.md
│   │   ├── incident-triage/
│   │   │   └── SKILL.md
│   │   └── done/
│   │       └── SKILL.md
│   └── hooks/
│       ├── protect-files.sh
│       ├── format.py
│       └── audit.py
├── src/
├── tests/
├── .github/
│   └── workflows/
│       └── ci.yml

```

B.1 Esimerkin työnkulku

1. Discovery

```
claude --permission-mode plan
```

2. Feature isolation

```
claude --worktree feature-auth
```

3. Parallel review

```
claude --resume reviewer-session
```

4. Headless CI triage

```
cat build.log | claude -p "Return JSON triage" --output-format json
```

5. Before merge

```
# /rewind if needed, then Git review + tests
```

```
# 6. Deploy only on explicit user action
/deploy staging
```

Liite C. Lähteet ja versiotiedot

Tämä opas käyttää kahta lähderyhmää: (A) käyttäjän toimittama videotranskriptio ja (B) nykyinen verkosta tarkistettu tekninen dokumentaatio. Transkriptiosta on säilytetty kaikki pyydetty pääaiheet ja käytännön demot; terminologiaa on päivitetty nykyisen Claude Code -dokumentaation kohdalla erikseen merkittynä.

Lähde	Käyttö
Käyttäjän videotranskriptio, 148 riviä	Perusrakenne, demo-ajatus, alkuperäiset esimerkit, terminologia
Claude Code - Features Overview	Laajennusmalli: Skills, Subagents, Hooks, MCP, agent teams
Claude Code - Subagents	Custom subagentit, tool limits, model selection
Claude Code - Skills	Skillit, frontmatter, invocation controls, arguments
Claude Code - Hooks	Hook-tyypit, lifecycle, matcherit, turvallisuus
Claude Code - MCP	MCP-asennus, HTTP, auth, output limits
Claude Code - Sessions	Resume, naming, branching, transcriptit
Claude Code - Worktrees	`--worktree`, `.worktreeinclude`, isolation
Claude Code - Headless	`-p`, `--bare`, output-format, scripts/CI
Claude Code - Checkpointing	Automatic checkpoints, `/rewind`
Model Context Protocol specification	Hosts, clients, servers, tools/resources/prompts
Granola Help Center - Granola MCP	Claude Code -komento ja OAuth

Lähdeviitteet: [S1] käyttäjän toimittama videotranskriptio; [S2] Features Overview; [S3] Subagents; [S4] Checkpointing; [S5] Sessions; [S6] Run agents in parallel; [S7] Configuration; [S8] Skills/Slash Commands; [S9] Quickstart; [S10] Permissions; [S11] Hooks reference; [S12] Hooks guide; [S13] MCP; [S14] Granola MCP; [S15] MCP specification; [S16] MCP server overview; [S17] MCP tools security; [S18] CLI reference; [S19] Worktrees; [S20] Plugin/Hook reference; [S21] Headless; [S22] Best Practices.

Keskeiset verkko-lähteet:

<https://code.claude.com/docs/en/features-overview>

<https://code.claude.com/docs/en/sub-agents>

<https://code.claude.com/docs/en/slash-commands>

<https://code.claude.com/docs/en/hooks>

<https://code.claude.com/docs/en/mcp>

<https://code.claude.com/docs/en/sessions><https://code.claude.com/docs/en/worktrees><https://code.claude.com/docs/en/headless><https://code.claude.com/docs/en/checkpointing><https://code.claude.com/docs/en/permissions><https://modelcontextprotocol.io/specification/2025-06-18/basic/index><https://help.granola.ai/article/granola-mcp>

Tarkistuspäivä: 8.9.2026. Claude Code ja sen CLI/agenttien ominaisuudet muuttuvat nopeasti; ennen tuotantokonfiguraation käyttöönottoa tarkista oman asennuksesi `claude --help`, CLI-reference ja kyseisen ominaisuuden oma virallinen dokumentaatio. Tämä on erityisen tärkeää hook-tapahtumille, lyhyille CLI-aliasmuodoille ja beta/versioriippuvaisille ominaisuuksille.

Liite D. Käytännön esimerkkien hakemisto

Oppaan käytännön esimerkit 01-35 kattavat vähintään vaaditun 30 esimerkin määrän. Ne on sijoitettu oikeisiin lukuihin, jotta niitä voidaan käyttää itsenäisinä harjoituksina.

Nro	Aihe
01	Ensimmäinen read-only työ
02	Kontekstin seuranta
03	Plan-mode analyysi
04	Headless työkalurajaus
05	Reviewer-agentti
06	Debugger-agentti
07	Security-agentti
08	Manuaalinen deploy skill
09	Skill-argumentti
10	Incident-triage skill
11	Formatter hook
12	Dangerous command hook
13	SessionStart + compact
14	MCP + issue tracker
15	MCP + observability
16	MCP + read-only DB
17	Riskialtis session branch
18	Dedicated bugfix session
19	PR session
20	Bugfix + feature worktrees
21	Subagent worktree
22	Worktree merge
23	CI build failure triage

Nro	Aihe
24	Headless read-only
25	Headless JSON
26	Broken build + rewind
27	Conversation-only rewind
28	Code-only rewind
29	TDD-agent workflow
30	Definition of Done skill
31	Review gate
32	Release pipeline
33	Production deploy gate
34	Secrets audit
35	Prompt injection test

Muista: Claude Code on tehokkaimmillaan silloin, kun probabilistinen agentti yhdistetään deterministiseen ympäröivään järjestelmään: Git, testit, permission rules, hooks, CI ja selkeä prosessi. Tavoite ei ole "antaa AI:lle kaikki oikeudet", vaan rakentaa sille hyvin rajattu ja samalla erittäin tehokas toimintaympäristö.